

# Reviewer Pack — Minimal Rank of Quasigroup Representations and Resolution Pr...

Entry 9c937c209e00 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 03:15:47 UTC

## Minimal Rank of Quasigroup Representations and Resolution Proof Width Correlation

**Entry ID:** 9c937c209e00 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 03:15:47 UTC

### 1. Conjecture

**Field A** (mathematical branch): Quasi-group Theory **Field B** (complexity object): Resolution Proof Complexity

#### Statement:

For every CNF  $\varphi$  with  $m$  clauses, the minimal rank of a quasigroup representation that can express all truth assignments of  $\varphi$  is linearly correlated with its resolution proof width  $w(\varphi)$ , such that  $\text{MinimalRank}(Q) = \Theta(w(\varphi))$  for any quasigroup  $Q$  that represents  $\varphi$ .

#### Rationale (proposer's reasoning):

Quasi-groups are algebraic structures that exhibit non-associative binary operations. The minimal rank of a quasigroup representation can be computed efficiently, making it a promising candidate for a complexity-theoretic invariant. A correlation between this rank and resolution proof width could provide insights into the structure of SAT instances.

**Taxonomy category:** quasi\_group\_theory\_resolution\_complexity (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 4e2cb7a0dd7391d0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient between minimal quasigroup representation rank and resolution proof width is  $\geq 0.8$  across all seeds, and no seed produces a correlation coefficient  $< 0.5$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 1.00       | SAFE             | SAFE             |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

## 4. Novelty audit

**Verdict:** NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - MinimalRank Quasi-group Theory Resolution Proof Complexity - Quasigroup representation Resolution proof width CNF - Resolution proof complexity Linear correlation MinimalRank Quasi-group

## 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i + max(range(i, rows), key=lambda x: abs(matrix[x][i]))
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            factor = Fraction(1, matrix[i][i])
            for j in range(cols):
                matrix[i][j] *= factor
            for k in range(rows):
                if k != i:
                    factor = -matrix[k][i]
                    for j in range(cols):
                        matrix[k][j] += factor * matrix[i][j]
        return matrix

    def rank(matrix):
        rows, cols = len(matrix), len(matrix[0])
        row_echelon_form = gaussian_elimination(matrix)
        rank = 0
        for i in range(rows):
            if any(row_echelon_form[i]):
                rank += 1
        return rank

    def resolution_width(cnf):
        # Simplified DPLL algorithm to compute resolution width
```

```

clauses = [set(clause) for clause in cnf]
queue = []
while True:
    unit_clause = next((c for c in clauses if len(c) == 1), None)
    if not unit_clause:
        return len(queue)
    literal, = unit_clause
    queue.append(literal)
    new_clauses = set()
    for clause in clauses:
        if literal in clause:
            continue
        if -literal in clause:
            return len(queue)
        new_clauses.add(clause ^ {literal})
    clauses.update(new_clauses)

def generate_cnf(num_vars, num_clauses):
    cnf = []
    literals = list(range(1, num_vars + 1)) + [-i for i in range(1, num_vars + 1)]
    for _ in range(num_clauses):
        clause = random.sample(literals, random.randint(2, 3))
        cnf.append(clause)
    return cnf

n_max = 40
instances_tested = 0
total_rank = 0
total_width = 0

for n in range(5, n_max + 1):
    for _ in range(6): # Ensure at least 30 instances per seed
        cnf = generate_cnf(n, random.randint(2 * n, 4 * n))
        rank_value = rank(cnf)
        width_value = resolution_width(cnf)
        total_rank += rank_value
        total_width += width_value
        instances_tested += 1

if instances_tested < 30:
    return {
        "metric_name": "Correlation Coefficient",
        "metric_value": None,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "Insufficient instances tested"
    }

mean_rank = total_rank / instances_tested
mean_width = total_width / instances_tested
correlation_coefficient = (instances_tested * sum(rank_value * width_value for rank_value, width_value in zip(cnf_ranks, cnf_widths)) - mean_rank * sum(cnf_widths)) / math.sqrt(
    instances_tested * sum((rank_value - mean_rank) ** 2 for rank_value in cnf_ranks) *
    instances_tested * sum((width_value - mean_width) ** 2 for width_value in cnf_widths))

```

```

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient >= 0.8 and all(0.5 <= coeff < 1 for coeff in [
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = list(map(int, sys.argv[1:])) or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + \
        [31, 37, 41, 43, 47, 53, 59, 61, 67, 71] + \
        [73, 79, 83, 89, 97, 101, 103, 107, 109, 113]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}\"}}")
        results.append(result)

    mean_metric_value = sum(result["metric_value"] for result in results if result["metric_value"] is not None) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results and result["metric_value"] >= 0.5):
        print("RESULT: FALSIFIED counterexample=\"Correlation coefficient < 0.8\" first_failing_seed={seed}")
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a51adce.py", line 118, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a51adce.py", line 80, in run_trial
    rank_value = rank(cnf)
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a51adce.py", line 38, in rank
    row_echelon_form = gaussian_elimination(matrix)
                       ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_0a51adce.py", line 28, in gaussian_elimination
    matrix[i][j] *= factor
    ~~~~~^
IndexError: list index out of range

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

**Reasoning:**

The test crashed before producing data, which means it did not complete its execution to calculate the correlation coefficients. | next: Re-run the test ensuring that it completes without crashing and produces the required correlation coefficient data.

## 11. Audit log (LLM calls)

**Total LLM calls:** 10

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 23744        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 15743        |
| 3  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 12076        |
| 4  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8285         |
| 5  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 8741         |
| 6  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17762        |
| 7  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 17763        |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 22564        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15372        |
| 10 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 19594        |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 161644 ms total latency. Provider mix: {'ollama\_remote': 10}

*(full prompt+response transcripts available in research/audit/9c937c209e00.jsonl)*

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/9c937c209e00.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** research/pvsnp\_sandbox/test\_\*.py (per-cycle)

**Replay tarball:** research/replay/9c937c209e00.tar.gz (if generated)